Lecture 28.09: Decision Tree Classifier in Python (Titanic Dataset)

📌 Topic Overview

In this hands-on coding lecture, we implement a Decision Tree Classifier using Scikit-Learn on the famous **Titanic dataset** (`sns.load_dataset('titanic')`), inspect missing values, encode features, train a baseline model, and observe severe overfitting.

📥 Dataset & Preprocessing Pipeline

The instructor loads the Titanic dataset from Seaborn and selects the core features:

- **Features (X)**: `pclass` , `sex` , `fare` , `embarked` , `age`
- **Target (y)**: `survived` (1 = Survived, 0 = Did not survive)

1. Handling Missing Data with `SimpleImputer`

- `age` : Imputed with the **median** value (`SimpleImputer(strategy="median")`).
- `embarked` : Imputed with the **most frequent** value (`SimpleImputer(strategy="most_frequent")`).

2. Encoding Categorical Features

- `sex` : Encoded to numerical (e.g., female=0, male=1).
- `embarked` : Encoded to numerical (e.g., C=0, Q=1, S=2).

⚠️ Training the Baseline Model: The Overfitting Trap

When we train an unconstrained `DecisionTreeClassifier(random_state=42)` :

- **Training Accuracy**: $\approx 98.2\%$ (Almost perfect memorization!)
- **Testing Accuracy**: $\approx 77.5\%$

The huge gap between training and testing performance clearly shows that an unpruned tree memorizes training data quirks rather than learning generalizable patterns.

💻 Complete Python Code from Lecture

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier

def train_titanic_baseline_tree():
    """
    Loads Titanic dataset, imputes missing values, and fits an unpruned
    DecisionTreeClassifier to demonstrate overfit in a real-world pipeline.
    """
    # 1. Load dataset
    df = sns.load_dataset('titanic')
    features = ['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']
    X = df[features].copy()
    y = df['survived']

    # 2. Encode categorical variables
    X['sex'] = X['sex'].map({'male': 0, 'female': 1})
    X['embarked'] = X['embarked'].map({'S': 0, 'C': 1, 'Q': 2})

    # 3. Impute missing values
    imputer_age = SimpleImputer(strategy='median')
    X['age'] = imputer_age.fit_transform(X[['age']])

    imputer_emb = SimpleImputer(strategy='most_frequent')
    X['embarked'] = imputer_emb.fit_transform(X[['embarked']])

    # 4. Train-Test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # 5. Fit unpruned Decision Tree
    clf = DecisionTreeClassifier(random_state=42)
    clf.fit(X_train, y_train)

    train_acc = clf.score(X_train, y_train)
    test_acc = clf.score(X_test, y_test)

    return clf, train_acc, test_acc

# --- Execute Function ---
tree_model, train_score, test_score = train_titanic_baseline_tree()
print(f"Train Accuracy: {train_score*100:.1f}%")
print(f"Test Accuracy:  {test_score*100:.1f}% (Overfitting Gap: {(train_score - te
```

🔑 Key Takeaways

1. Real-world datasets require proper missing value handling (`SimpleImputer`) and categorical encoding before fitting trees.
2. An unconstrained decision tree achieves near 100% training accuracy but drops significantly on test data.
3. Tree visualization (`plot_tree`) reveals an overly deep, complex tree that requires pruning.